Task 2.1 - Prompting Strategy for Code Generation

Description

Select one self-contained group of requirements (i.e. requirements that can be implemented independently from other groups) consisting of at least six (6) functional requirements and two (2) non-functional requirements. This group should represent a meaningful feature set of the system (for example, AI-usage logging or user authentication) and be suitable for isolated design and implementation.

Example scope of the requirements:

- *Functional requirement (user story):* As a student, I want to log how I used an AI tool for an assignment, so that my AI usage is documented and transparent.
- *Non-functional requirement (user story - usability):* As a student, I want the AI-usage logging interface to be simple and quick to use, so that logging does not interrupt my study workflow.

This task focuses on designing a structured prompting strategy for building components of the AIGuidebook website using AI-assisted code development tools (e.g. ChatGPT, GitHub Copilot, or similar). NTNU provides GitHub Copilot licences for students, and the InnSpill platform offers free access; additional tools may require personal licences.

Using the requirements and dependency map from Task 1.3, you will design a systematic prompting approach that supports:

- Requirement-driven code generation
- Component-level architectural planning
- Iterative refinement and debugging
- Effective human-AI co-development

The goal is not to write code, but to define a repeatable prompting strategy that enables developers to reliably generate and validate React components aligned with real system requirements. Your strategy must demonstrate understanding of a full-stack web architecture, for example React for frontend, and Node.js + Express for backend.

Expected Outcome

Prompt Strategy Document (maximum 500 words) describing a prompting approach and patterns used.

Evaluation Criteria

- **Quality of Prompt Strategy:** The strategy is coherent, repeatable, and covers known prompting patterns (at least 2).
- **Technical Understanding of full-stack web development:** Clear awareness of a full-stack web project structure, component organisation, and architectural concerns.

Our Submission

Selected Requirements

We select the **Logging & Risk Engine** group: RE-09 through RE-16 (8 functional requirements) plus NFR-01 (logging responsiveness < 2s) and NFR-02 (reply time < 30s). This group covers a self-contained pipeline from usage logging through risk classification and advisor alerting, suitable for isolated design and testing.

Prompting Approach

Our strategy uses three established prompting patterns, **Prompt Chaining**, **Iterative Refinement**, and **Few-Shot Prompting**, organized into a test-driven development pipeline. The AI's instruction file (CLAUDE.md) is continuously monitored and self-checked for updates as conventions emerge across phases.

Phase 1: Architecture Scoping & Scaffolding

The AI receives the complete requirement set alongside domain context describing the AIGuidebook system. The prompt asks it to analyze the requirements and propose a full-stack architecture, justifying each choice against specific requirements. For example, NFR-01's 2-second constraint informs database selection, and RE-14's declaration-versus-log comparison motivates a relational model. Critically, the project is **scaffolded using standard CLI tooling** (e.g., `bunx create-hono`, `bunx create-vite`) rather than having the AI generate boilerplate from scratch. This ensures a known-good foundation. The AI guides which commands to run and how to configure the result. The output - a working skeleton with the chosen stack - becomes the input context for all subsequent prompts (**prompt chaining**).

Phase 2: Test Specification (Red)

Processing **one requirement at a time**, the AI writes a single end-to-end backend test that verifies the requirement. The prompt includes the architecture from Phase 1 and the full requirement text. The test must fail because no implementation exists yet, and we confirm the failure before advancing. After the first requirement (RE-09) is fully implemented and tested, its completed test-and-implementation pair serves as a **few-shot example** in prompts for subsequent requirements, showing the AI the expected format and quality.

Phase 3: Minimal Implementation (Green)

For the same requirement, the AI receives the failing test and is prompted to write the minimal backend code (route handler, service logic, data model) to make it pass. If the test still fails, error output is fed back to the AI in a feedback loop (**iterative refinement**) until the test passes. Only then do we return to Phase 2 for the next requirement.

The first requirement (RE-09) is completed sequentially to establish the pattern. Once proven, remaining requirements can be dispatched to **parallel agent teams**, each running its own red-green cycle with single-test discipline.

Phase 4: Refactor and Integration

After all requirements have passing tests, the AI reviews the codebase for refactoring opportunities and non-functional performance targets. ESLint enforces code quality throughout; AI-generated code must pass linting before advancing. All existing tests must continue to pass (**iterative refinement**).

Phase 5: Frontend

Once all backend tests are green, TypeScript types are generated from the OpenAPI spec (`openapi-typescript`). The frontend is then built using `openapi-fetch` as a type-safe API client, consuming the

verified endpoints. This ordering is deliberate: backend tests are fast and enable rapid feedback during AI-assisted development, while the frontend benefits from a stable, documented API.

Feedback

Not yet graded.

---

task: "2.2" title: Code Generation course: TDT4242 group: 17 deadline: 2026-03-02T23:59:00 score: null status: not-submitted

Task 2.2 - Code Generation

Description

Use ChatGPT, Claude, GitHub Copilot, or similar AI tools to generate a full-stack web application that implements the requirements selected from Exercise 2.1. Following your defined prompting strategy and requirement set, you will guide an AI code assistant to generate the initial version of the system.

Your objective is to deliver:

- Correct installation and configuration of all required libraries and dependencies
- Revision of AI-generated code if needed
- A working full stack web application that runs locally in your computer
- A clear and complete explanation of the codebase structure and design decisions

Expected Outcome

A document including:

- Link to a public GitHub repository containing the full project
- Screenshots demonstrating successful execution
- Explanation of the role of each major component
- What code was generated by AI and what was written or modified by you

Evaluation Criteria

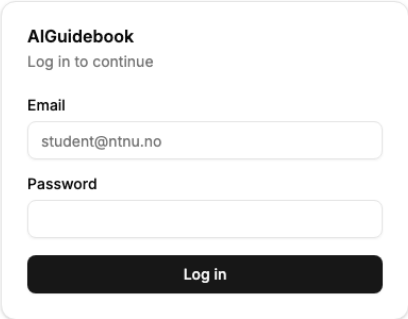
- **Successful Code Execution:** The code must run without errors in your local environment and meet the functional scope agreed.
- **Quality of reflection:** Students must clearly explain architecture, component purpose, state management choices, and division between AI-produced and student-written code.

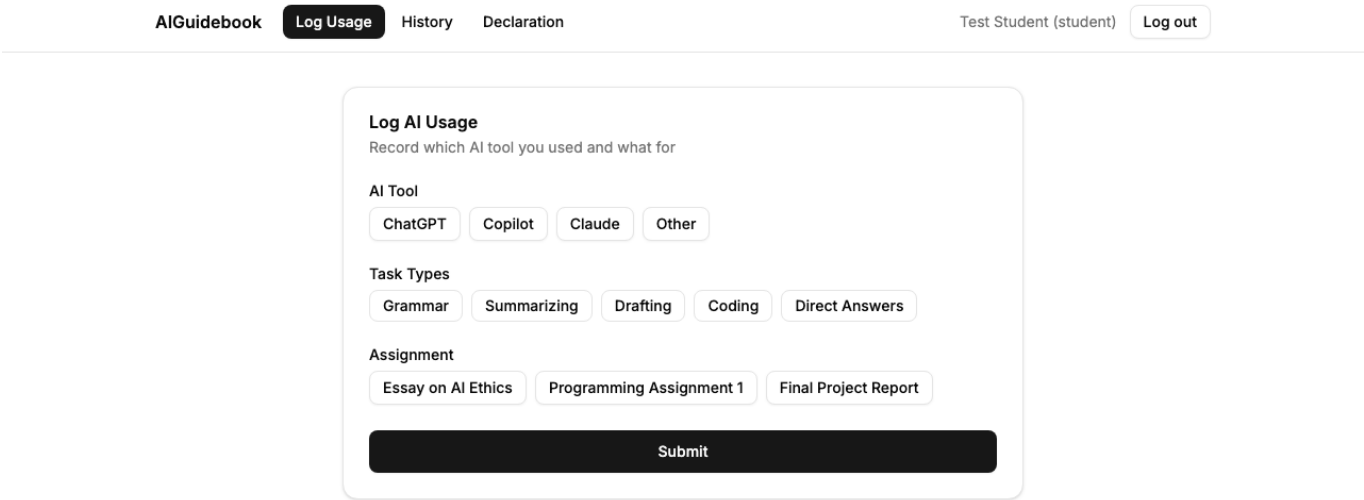
Our Submission

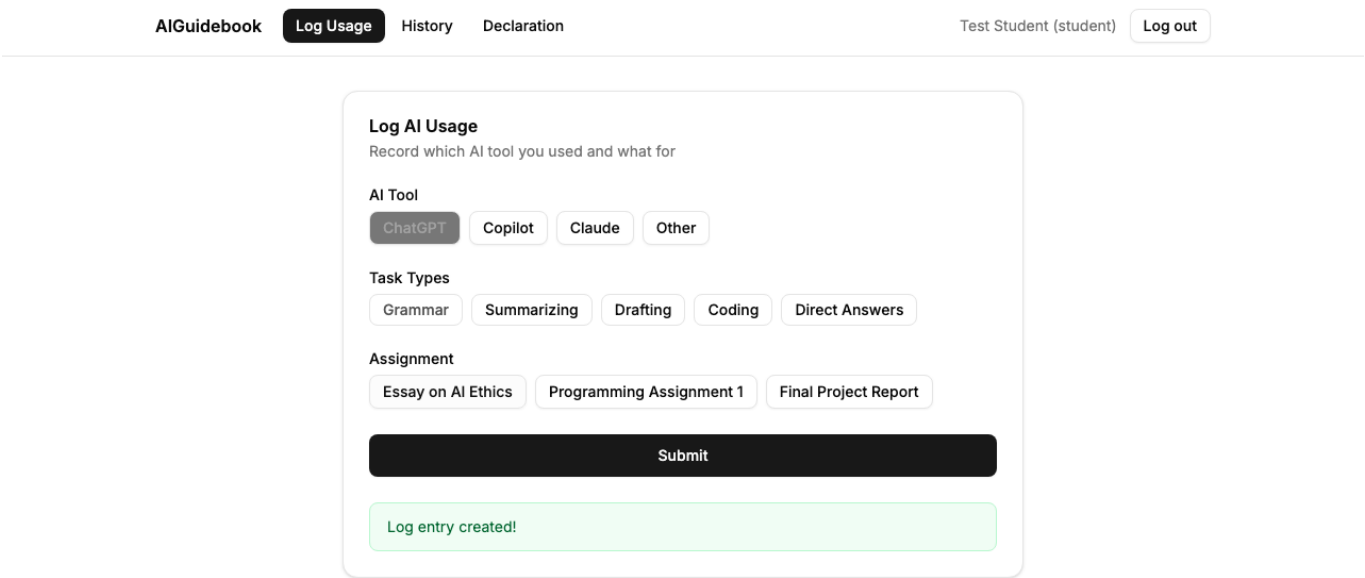
Repository

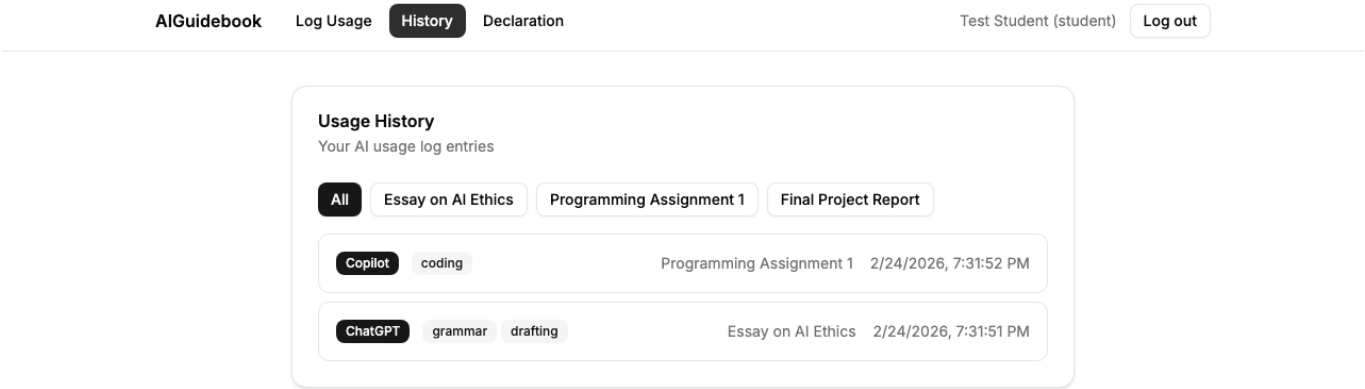
github.com/ryanravn/tdt4242-ntnu-group-17-assignment

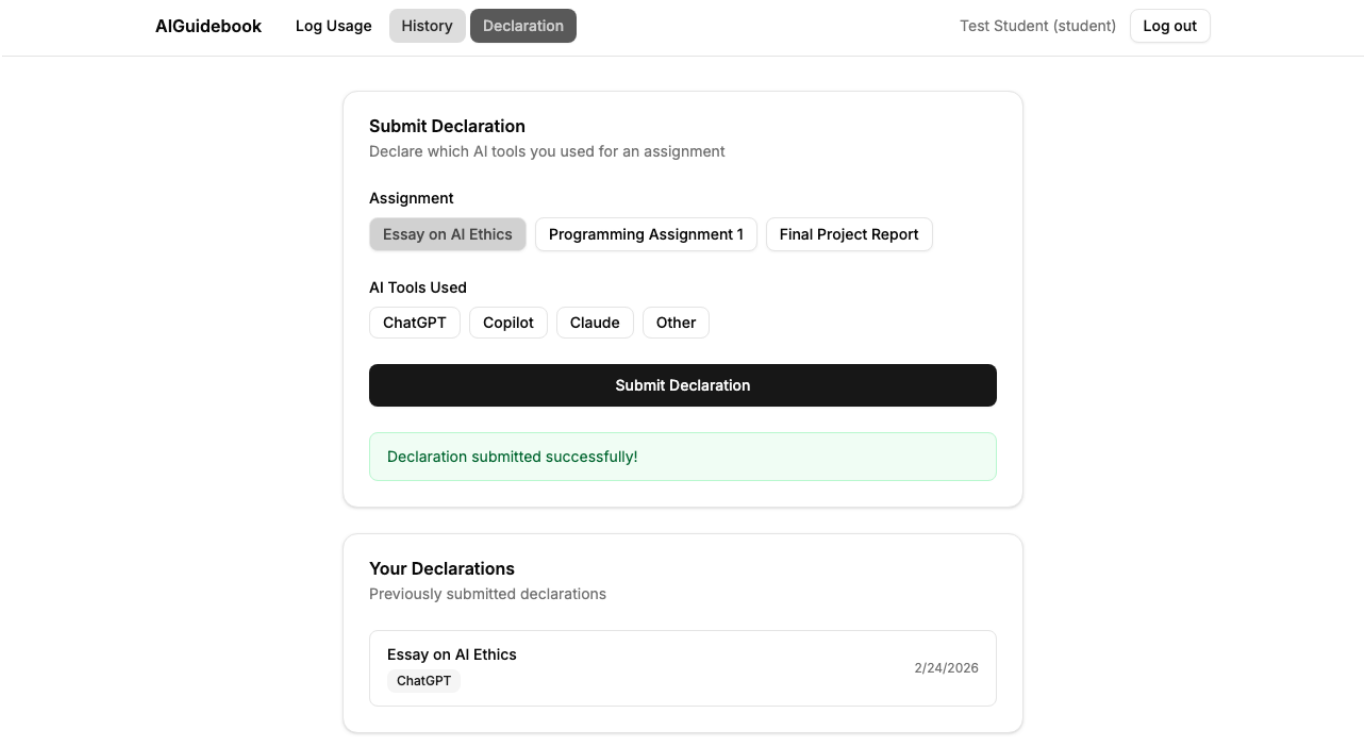
Screenshots

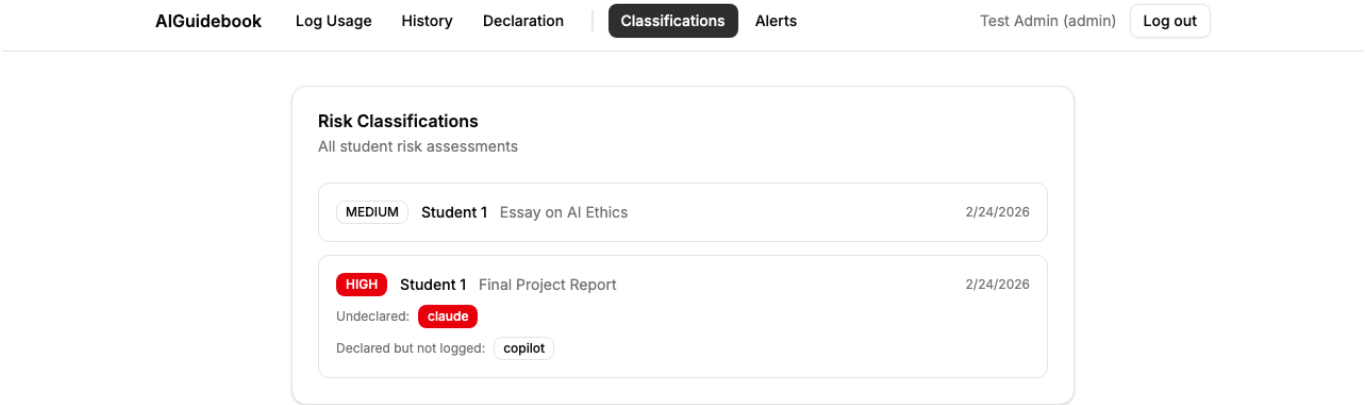

AIGuidebook
Log in to continue
Email
student@ntnu.no
Password
Log in

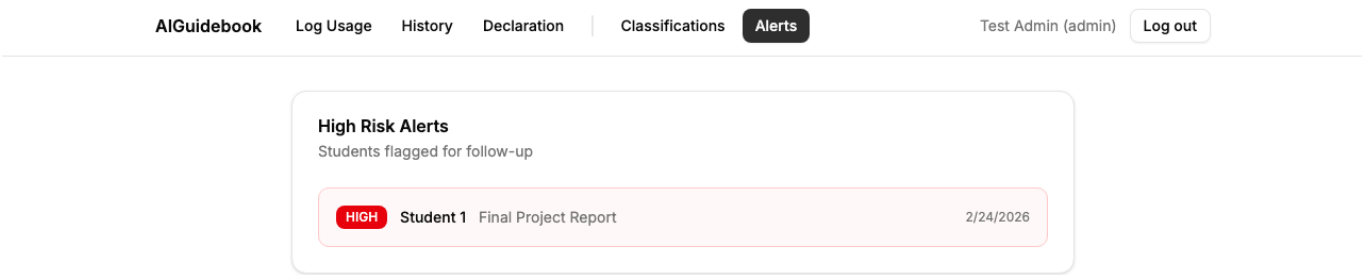

AIGuidebook
Log Usage
History
Declaration
Test Student (student)
Log out
Log AI Usage
Record which AI tool you used and what for
AI Tool
ChatGPT
Copilot
Claude
Other
Task Types
Grammar
Summarizing
Drafting
Coding
Direct Answers
Assignment
Essay on AI Ethics
Programming Assignment 1
Final Project Report
Submit


AIGuidebook
Log Usage
History
Declaration
Test Student (student)
Log out
Log AI Usage
Record which AI tool you used and what for
AI Tool
ChatGPT
Copilot
Claude
Other
Task Types
Grammar
Summarizing
Drafting
Coding
Direct Answers
Assignment
Essay on AI Ethics
Programming Assignment 1
Final Project Report
Submit
Log entry created!


AIGuidebook
Log Usage
History
Declaration
Test Student (student)
Log out
Usage History
Your AI usage log entries
All
Essay on AI Ethics
Programming Assignment 1
Final Project Report
Copilot
coding
Programming Assignment 1
2/24/2026, 7:31:52 PM
ChatGPT
grammar
drafting
Essay on AI Ethics
2/24/2026, 7:31:51 PM


AIGuidebook
Log Usage
History
Declaration
Test Student (student)
Log out
Submit Declaration
Declare which AI tools you used for an assignment
Assignment
Essay on AI Ethics
Programming Assignment 1
Final Project Report
AI Tools Used
ChatGPT
Copilot
Claude
Other
Submit Declaration
Declaration submitted successfully!
Your Declarations
Previously submitted declarations
Essay on AI Ethics
ChatGPT
2/24/2026


AIGuidebook
Log Usage
History
Declaration
Classifications
Alerts
Test Admin (admin)
Log out
Risk Classifications
All student risk assessments
MEDIUM
Student 1
Essay on AI Ethics
2/24/2026
HIGH
Student 1
Final Project Report
2/24/2026
Undeclared:
claude
Declared but not logged:
copilot



Codebase Structure and Component Roles

The application is a single-project full-stack web app. Hono serves the REST API and, in production, the built SolidJS frontend from one process. The project is split into four top-level directories.

Backend (`server/`)

File	Role
`server/index.ts`	Application entry point. Creates an `OpenAPIHono` instance, registers CORS, the JSON error handler, all route modules, the OpenAPI spec endpoint (`/openapi`), Scalar API docs (`/docs`), and a health check. In production it serves the built frontend as static files; in development it redirects to the Vite dev server.
`server/routes/auth.ts`	Authentication routes (`POST /login`, `GET /me`). Login verifies hashed passwords with `Bun.password.verify`, issues a JWT, and returns the user. `/me` validates the JWT and returns the current user.
`server/routes/logs.ts`	Logging routes (RE-09, RE-10, RE-12). `POST /` creates a log entry after validating the assignment exists (RE-10). `GET /` returns the student's logs ordered by most-recent-first, with optional `assignment_id`, `from`, and `to` query filters.

File	Role
`server/routes/declarations.ts`	Declaration route (RE-11) and the risk classification engine (RE-13, RE-14, RE-15, RE-16). `POST /` inserts the declaration (returning 409 if one already exists for the same student+assignment), then calls `classifyDeclaration()`. That function fetches all logs for the student+assignment, compares logged tools against declared tools (RE-14), determines risk level (RE-13), inserts a classification record, and creates an alert if the risk is high (RE-16). `GET /` returns the student's own declarations.
`server/routes/classifications.ts`	Read-only admin endpoint. `GET /` returns all stored risk classifications with risk level, discrepancy details, and timestamps.
`server/routes/alerts.ts`	Read-only admin endpoint. `GET /` returns all high-risk alert records.
`server/lib/auth.ts`	JWT helper functions: `createToken` signs a payload with HS256, `requireAuth` extracts and verifies the Bearer token from the Authorization header.
`server/db/schema.ts`	Drizzle ORM schema defining all tables (`users`, `assignments`, `logs`, `declarations`, `classifications`, `alerts`) and enums (`role`, `tool`, `task_type`, `risk_level`). Foreign keys enforce referential integrity. The declarations table has a composite unique constraint on `(studentId, assignmentId)`.
`server/db/client.ts`	Creates the Drizzle client from `DATABASE_URL` using `postgres` and exports `db`.
`server/db/seed.ts`	Seeds two users (student and admin with hashed passwords) and three assignments. Uses `onConflictDoNothing()` for idempotency.

Frontend (`client/`)

File	Role
`client/index.tsx`	SolidJS entry point. Renders `<App />` into the DOM.
`client/App.tsx`	Root component. Manages authentication state (login form, session restore, dev auto-login with seed user). After login, renders a navigation bar (role-aware: students see Log Usage, History, Declaration; admins also see Classifications and Alerts) and a `<Switch>` that mounts the active page component based on a signal.

File	Role
`client/lib/api.ts`	Thin fetch wrapper. `apiFetch<T>()` attaches the JWT from localStorage to every request and throws on non-OK responses. The `api` object exposes typed methods (`createLog`, `getLogs`, `createDeclaration`, `getDeclarations`, `getClassifications`, `getAlerts`) so pages don't deal with raw fetch.
`client/lib/utils.ts`	`cn()` utility - merges Tailwind classes using `clsx` + `tailwind-merge`. Used by all components.
`client/pages/LogUsage.tsx`	Log AI Usage form (RE-09). Lets the student pick an AI tool, toggle one or more task types, select an assignment, and submit. Calls `api.createLog()`.
`client/pages/UsageHistory.tsx`	Usage History view (RE-12). Fetches logs on mount via `api.getLogs()`, displays them in a list with assignment-filter buttons.
`client/pages/SubmitDeclaration.tsx`	Declaration form (RE-11). Select an assignment and toggle which tools were used, then submit. Shows existing declarations below the form (fetched via `api.getDeclarations()`). Displays 409 errors as "Declaration already exists".
`client/pages/Classifications.tsx`	Admin classifications view (RE-15). Fetches all classifications on mount, displays risk level as a colored badge, and lists any undeclared or declared-not-logged tool discrepancies.
`client/pages/Alerts.tsx`	Admin alerts view (RE-16). Fetches all alerts on mount, displays each with a destructive badge, student/assignment info, and timestamp.
`client/components/*`	Reusable UI primitives (Button, Input, Label, Card, Badge, Sonner toast) built with Kobalte for accessibility, CVA for variant management, and Tailwind for styling. Follows the same shadcn-style pattern used in our reference project.
`client/index.css`	Tailwind 4 theme with OKLCH color variables, Inter font, light/dark mode support, and base styles.

Tests (`tests/`)

File	Covers
`tests/health.test.ts`	Health check and OpenAPI spec endpoints
`tests/auth.test.ts`	Login (valid, wrong password, non-existent user, missing fields) and /me (authenticated, no token, invalid token)

File	Covers
`tests/logs.test.ts`	RE-09 (log creation, validation errors, auth) and RE-10 (assignment validation, 404 for bad ID)
`tests/logs-read.test.ts`	RE-12 (list logs, ordering, assignment_id filter, date range filter, empty array, auth)
`tests/declarations.test.ts`	RE-11 (create declaration, 409 conflict, validation, auth, auto-triggers classification)
`tests/risk.test.ts`	RE-13 (low/medium/high classification), RE-14 (undeclared tools, declared-not-logged, no discrepancies), RE-15 (GET /api/classifications), RE-16 (alert on high risk, no alert on low/medium, GET /api/alerts)

All 38 tests use `app.request()` directly against the Hono app - no HTTP server, no generated client. Each test file cleans its data with `beforeAll` to ensure isolation.

Configuration and Tooling

File	Role
`vite.config.ts`	Configures SolidJS and Tailwind plugins, the `@` path alias to `client/`, proxies `/api`, `/openapi`, and `/docs` to localhost:3000 in development, and builds to `dist/client`.
`drizzle.config.ts`	Points Drizzle Kit at the schema file and PostgreSQL connection for migrations.
`scripts/generate-api-types.ts`	Imports the Hono app in-process, fetches the `/openapi` spec, and pipes it through `openapi-typescript` to generate `client/lib/api-types.d.ts`.
`scripts/watch-api-types.ts`	Polls the OpenAPI spec during development and regenerates types when the schema changes.
`eslint.config.js`	ESLint with TypeScript plugin. No Prettier - linting only.
`docker-compose.yml`	PostgreSQL 17 container on port 5432. Works with both Docker and Podman.
`CLAUDE.md`	AI instruction file linking key documents, describing the stack, TDD workflow, development order, project layout, commands, and conventions. Continuously updated as conventions emerged.

AI-Generated vs. Human-Written Code

All application code was generated by AI (Claude, via Claude Code CLI). This includes:

- All backend route handlers, database schema, seed data, and auth logic
- All frontend pages, components, CSS theme, and API client
- All 38 test files
- Configuration files (Vite, Drizzle, ESLint, Docker Compose, type generation scripts)

Human contributions were limited to:

- Technology stack selection and architecture decisions (`decisions.md`)
- Prompting strategy and phasing (`tasks/2.1-prompting-strategy.md`)
- The AI instruction file (`CLAUDE.md`) - initial structure and ongoing refinements
- Requirements revision (`REQUIREMENTS.md`) - rewriting abstract requirements into concrete, testable acceptance criteria with specific API endpoints and HTTP status codes
- Reviewing AI output and providing corrective feedback (e.g. "use Bun not Node", "no monorepo", "use Podman", "auto-login in dev")

No application code (TypeScript, TSX, CSS) was manually written or edited by a human. All code changes went through the AI, guided by human review and iterative feedback as described in the prompting strategy.

Feedback

Not yet graded.

---

task: "2.3" title: Code Review course: TDT4242 group: 17 deadline: 2026-03-02T23:59:00 score: null status: not-submitted

Task 2.3 - Code Review

Description

In this task, you will analyse, review, and improve the quality of the codebase produced in Task 2.2. You will apply code review practices to identify and reason about code smells—structural weaknesses that can reduce maintainability, readability, reliability, or performance. You must use at least two (2) of the following review approaches: Manual code review Peer review with classmates AI-assisted code analysis tools (e.g. ChatGPT, GitHub Copilot, SonarLint) Examples of code smells include (but are not limited to): long or overly complex components, duplicated logic, primitive obsession, large classes, long parameter lists, temporary fields, shotgun surgery, dead or unreachable code, excessive comments, deeply nested JSX, mixing UI and business logic, and missing error handling. If peer review is used, participating groups must agree on the collaboration in advance, and the codebase must be made available for review one week before the submission deadline. If AI tools are used, you must: Include sample prompts and responses Clearly state which suggestions were accepted or rejected Explain the reasoning behind these decisions

Expected Outcome

Code Review Summary Document that: Describes the review method used in sufficient detail to be repeatable Lists identified code smells, including: (1) Name of the smell (2)Short description (3) Identify method (4) Relevant original code snippets

Evaluation Criteria

Coverage: demonstrates broad and systematic coverage of the codebase, including components, pages, utilities, and data flow. Quality of code smell identification: Focus on meaningful structural issues rather than

superficial style problems Documented review methodology: The review process is clearly described and reproducible.

Review Methodology

Two review approaches were used: **AI-assisted code analysis** and **manual code review**.

Method 1: AI-Assisted Code Analysis (Claude via Cursor)

Tool: Claude (claude-4.6-opus) running inside Cursor IDE with full file access to the repository.

Process:

1. Prompt Claude with a request to systematically review every source file in the project (`server/`, `client/`, `tests/`).
2. Claude reads each file in full and analyses it for structural issues.
3. Findings are returned as a list of code smells with name, description, location, and code snippets.
4. The developer reviews each finding, deciding whether to accept or reject it, with reasoning.

Reproducibility: Any developer can replicate this by opening the project in Cursor, selecting all source files, and prompting: "*Review these files for code smells. For each smell, provide: (1) the name, (2) a short description, (3) file and line numbers, (4) the relevant code snippet.*"

Sample prompt:

> Read all source files under `server/`, `client/pages/`, `client/lib/`, and `tests/`. Identify structural code smells — duplicated logic, large functions, magic numbers, missing abstractions, mixing concerns, etc. For each smell, provide the name, a short description, the file and line numbers, and the original code snippet.

Sample response (excerpt):

> **Duplicated response mapping** — Every route handler manually maps DB rows to JSON with the same `{ id: e.id, ..., createdAt: e.createdAt.toISOString() }` pattern. Found in `server/routes/logs.ts` (lines 81–89, 134–141), `server/routes/declarations.ts` (lines 146–153, 181–186), `server/routes/classifications.ts` (lines 42–51), `server/routes/alerts.ts` (lines 42–50). A shared `toResponse()` mapper per entity would remove this duplication.

Decisions on AI suggestions:

#	AI Suggestion	Decision	Reasoning
1	Duplicated response mapping across routes	**Accepted**	Verified: 4 route files repeat the same map-to-JSON pattern
2	God function in `classifyDeclaration()`	**Accepted**	Function handles 4 requirements, mixes DB queries with business logic and side effects
3	Magic number `>= 3` for medium risk threshold	**Accepted**	Threshold has no named constant, making it hard to discover or change

#	AI Suggestion	Decision	Reasoning
4	Missing admin role check on classifications/alerts	**Accepted**	Verified: `requireAuth()` only checks for a valid JWT, never checks `role`
5	Primitive obsession in Zod schemas	**Accepted**	`declaredTools` accepts any string array despite having a `toolEnum` in the DB schema
6	Hardcoded ASSIGNMENTS duplicated across pages	**Accepted**	Confirmed in 5 separate files with two different shapes (array vs Record)
7	Duplicated TOOL_LABELS across pages	**Accepted**	Same mapping in 3 files
8	Duplicated success/error message UI blocks	**Accepted**	Identical styled divs in LogUsage and SubmitDeclaration
9	Duplicated loading/error/empty state pattern	**Accepted**	Same trio of `<Show>` blocks in Classifications, Alerts, UsageHistory
10	Large component mixing concerns in App.tsx	**Accepted**	209-line component combines auth, routing, navigation, and login form
11	Double fetch on mount in UsageHistory	**Accepted**	`onMount` + `createEffect` both call `fetchLogs()` on initial render
12	Duplicated `login()` helper in test files	**Accepted**	Identical function in all 5 test files
13	Inconsistent test isolation	**Accepted**	Only 3 of 5 test files use `beforeAll` cleanup
14	Inline type definitions instead of shared types	**Accepted**	Types like `LogEntry`, `Classification`, `Alert` redefined in each file

Method 2: Manual Code Review

Process:

1. Read each source file top-to-bottom, checking for: separation of concerns, error handling, duplication, naming, data flow, and security.
2. Cross-reference files to find patterns repeated across modules.
3. Validate and enrich the AI findings with concrete impact analysis.

Reproducibility: Open each file in the IDE, read sequentially, and use project-wide search (`grep` / `rg`) to confirm duplication across files. Compare Zod schemas against the Drizzle schema to check for type mismatches.

Code Smell Catalogue

Smell 1: Duplicated Response Mapping

Name: Duplicated Code

Description: Every route handler manually maps database rows to JSON response objects using the same pattern: destructure fields and call `.toISOString()` on the timestamp. This logic is repeated in 4 route files (6 occurrences total) with no shared utility function.

Identification method: AI-assisted analysis, confirmed by manual cross-file comparison.

Code snippets:

`server/routes/logs.ts` — POST handler (lines 81–89):

```
return c.json(
  {
    id: entry.id,
    tool: entry.tool,
    taskTypes: entry.taskTypes,
    assignmentId: entry.assignmentId,
    createdAt: entry.createdAt.toISOString(),
  },
  201,
);
```

`server/routes/logs.ts` — GET handler (lines 134–141):

```
return c.json(
  entries.map((e) => ({
    id: e.id,
    tool: e.tool,
    taskTypes: e.taskTypes,
    assignmentId: e.assignmentId,
    createdAt: e.createdAt.toISOString(),
  })),
  200,
);
```

`server/routes/declarations.ts` — POST handler (lines 146–153):

```
return c.json(
  {
    id: declaration.id,
    assignmentId: declaration.assignmentId,
    declaredTools: declaration.declaredTools,
    createdAt: declaration.createdAt.toISOString(),
  },
  201,
);
```

`server/routes/classifications.ts` — GET handler (lines 42–51):

```
return c.json(
  entries.map((e) => ({
    id: e.id,
    studentId: e.studentId,
    assignmentId: e.assignmentId,
    riskLevel: e.riskLevel,
    undeclaredTools: e.undeclaredTools,
    declaredNotLogged: e.declaredNotLogged,
    createdAt: e.createdAt.toISOString(),
  })),
  200,
);
```

Impact: Adding a new field (e.g. `updatedAt`) requires changing every handler. A shared `toLogResponse()`, `toDeclarationResponse()`, etc. would centralise the mapping.

Smell 2: God Function

Name: Long Method / God Function

Description: The `classifyDeclaration()` function in `server/routes/declarations.ts` (lines 24–86) is a 62-line function that handles four separate requirements (RE-13 risk classification, RE-14 declaration vs log comparison, RE-15 auto-classify trigger, RE-16 alert creation). It mixes database queries, business logic (tool comparison, risk determination), record insertion, and conditional side effects (alert creation) in a single function with no separation of concerns.

Identification method: AI-assisted analysis.

Code snippet:

`server/routes/declarations.ts` (lines 24–86):

```
async function classifyDeclaration(
  studentId: number,
  assignmentId: number,
  declaredTools: string[],
) {
  // RE-14: Get all logs for this student+assignment
  const logEntries = await db
    .select()
    .from(logs)
    .where(and(eq(logs.studentId, studentId), eq(logs.assignmentId,
assignmentId)));

  // RE-14: Compare logged tools vs declared tools
  const loggedTools = [...new Set(logEntries.map((l) => l.tool))];
  const undeclaredTools = loggedTools.filter((t) => !declaredTools.includes(t));
  const declaredNotLogged = declaredTools.filter((t) => !loggedTools.includes(t));
```

```
  // RE-13: Determine risk level
  let riskLevel: "low" | "medium" | "high" = "low";

  const hasDirectAnswers = logEntries.some((l) =>
    l.taskTypes.includes("direct_answers"),
  );
  const hasUndeclaredTools = undeclaredTools.length > 0;

  if (hasDirectAnswers || hasUndeclaredTools) {
    riskLevel = "high";
  } else {
    const hasSubstantialTasks = logEntries.some(
      (l) => l.taskTypes.includes("drafting") || l.taskTypes.includes("coding"),
    );
    const hasHighFrequency = logEntries.length >= 3;

    if (hasSubstantialTasks || hasHighFrequency) {
      riskLevel = "medium";
    }
  }

  // Insert classification record
  const [classification] = await db
    .insert(classifications)
    .values({
      studentId,
      assignmentId,
      riskLevel,
      undeclaredTools,
      declaredNotLogged,
    })
    .returning();

  // RE-16: If high risk, create an alert
  if (riskLevel === "high") {
    await db.insert(alerts).values({
      classificationId: classification.id,
      studentId,
      assignmentId,
      riskLevel: "high",
    });
  }

  return classification;
}
```

Impact: This function cannot be unit-tested in isolation — testing the risk algorithm requires a full database round trip. Extracting the comparison logic and risk determination into pure functions would improve testability and readability.

Smell 3: Magic Number

Name: Magic Number

Description: The medium-risk frequency threshold `logEntries.length >= 3` in the risk classification logic has no named constant. The number 3 carries business meaning ("3 or more log entries = high frequency") but is embedded directly in a conditional expression.

Identification method: AI-assisted analysis.

Code snippet:

`server/routes/declarations.ts` (line 56):

```
const hasHighFrequency = logEntries.length >= 3;
```

Impact: If the risk threshold changes, a developer must search through business logic to find the number. A named constant like `const HIGH_FREQUENCY_THRESHOLD = 3` would make the intent clear and the value easy to change.

Smell 4: Missing Role-Based Authorization

Name: Missing Error Handling / Security Defect

Description: The `GET /api/classifications` and `GET /api/alerts` endpoints are intended for administrators only (per RE-15, RE-16), but the route handlers only call `requireAuth()` which validates the JWT without checking the user's role. Any authenticated student can access all classification and alert data.

Identification method: Manual code review — cross-referencing requirements (RE-15: "an administrator calls GET /api/classifications") against the implementation.

Code snippets:

`server/routes/classifications.ts` (lines 37–39):

```
async (c) => {
    await requireAuth(c);
    const entries = await db.select().from(classifications);
```

`server/routes/alerts.ts` (lines 37–39):

```
async (c) => {
    await requireAuth(c);
    const entries = await db.select().from(alerts);
```

Compare with `server/lib/auth.ts` (lines 11–24) — `requireAuth` returns the payload but the role is never checked:

```
export async function requireAuth(c: Context) {
  const header = c.req.header("Authorization");
  if (!header?.startsWith("Bearer ")) {
    throw new HTTPException(401, { message: "Missing or invalid token" });
  }

  const token = header.slice(7);
  try {
    const payload = await verify(token, JWT_SECRET, "HS256");
    return payload as { sub: number; role: string };
  } catch {
    throw new HTTPException(401, { message: "Invalid token" });
  }
}
```

Impact: A student can view all risk classifications and alerts for all students by calling the admin endpoints directly. The fix would be to add a `requireAdmin()` middleware or check `payload.role === "admin"` in the handler.

Smell 5: Primitive Obsession

Name: Primitive Obsession

Description: The Zod request schemas for `declaredTools` and `taskTypes` use `z.array(z.string())`, accepting any arbitrary string. However, the database schema defines strict enums (`toolEnum` with values `chatgpt | copilot | claude | other` and `taskTypeEnum` with values `grammar | summarizing | drafting | coding | direct_answers`). The validation layer does not enforce the same constraints as the data layer.

Identification method: AI-assisted analysis, confirmed by manual comparison of Zod schemas vs Drizzle schema.

Code snippets:

`server/routes/declarations.ts` (lines 10–13) — accepts any string:

```
const createDeclarationSchema = z.object({
  assignmentId: z.number(),
  declaredTools: z.array(z.string()).min(1),
});
```

`server/db/schema.ts` (lines 7–9) — defines strict enums:

```
export const toolEnum = pgEnum("tool", ["chatgpt", "copilot", "claude", "other"]);
export const taskTypeEnum = pgEnum("task_type", ["grammar", "summarizing",
"drafting", "coding", "direct_answers"]);
```

The `logs.ts` route does use `z.enum()` for the `tool` field but not for `taskTypes`:

```
const createLogSchema = z.object({
  tool: z.enum(["chatgpt", "copilot", "claude", "other"]),
  taskTypes: z.array(z.string()).min(1),
  assignmentId: z.number(),
});
```

Impact: A student could submit a log with `taskTypes: ["anything"]` or a declaration with `declaredTools: ["nonexistent"]`. The database may accept it (the `declared_tools` column is `text[]`, not using the enum) but the risk classification logic would not recognise non-standard values, producing unreliable results.

Smell 6: Duplicated Constant — ASSIGNMENTS

Name: Duplicated Code

Description: The hardcoded list of assignments appears in 5 separate frontend files with two different shapes: as an array of `{ id, title }` objects in `LogUsage.tsx` and `SubmitDeclaration.tsx`, and as a `Record<number, string>` in `UsageHistory.tsx`, `Classifications.tsx`, and `Alerts.tsx`. The data is also defined in `server/db/seed.ts` — the frontend duplicates must be kept manually in sync.

Identification method: AI-assisted analysis, confirmed by searching for `ASSIGNMENTS` across the codebase.

Code snippets:

`client/pages/LogUsage.tsx` (lines 37–41) — array shape:

```
const ASSIGNMENTS = [
  { id: 1, title: "Essay on AI Ethics" },
  { id: 2, title: "Programming Assignment 1" },
  { id: 3, title: "Final Project Report" },
];
```

`client/pages/UsageHistory.tsx` (lines 21–25) — Record shape:

```
const ASSIGNMENTS: Record<number, string> = {
  1: "Essay on AI Ethics",
  2: "Programming Assignment 1",
  3: "Final Project Report",
};
```

`client/pages/Alerts.tsx` (lines 21–25) — same Record shape, duplicated:

```
const ASSIGNMENTS: Record<number, string> = {
  1: "Essay on AI Ethics",
  2: "Programming Assignment 1",
  3: "Final Project Report",
};
```

Impact: Adding or renaming an assignment requires editing 5 frontend files plus the seed script. A shared `client/lib/constants.ts` or an API endpoint that returns assignments would eliminate this duplication.

Smell 7: Duplicated Constant — TOOL_LABELS

Name: Duplicated Code

Description: The `TOOL_LABELS` mapping from tool identifiers to display names is defined identically in three separate page components.

Identification method: AI-assisted analysis.

Code snippets:

`client/pages/LogUsage.tsx` (lines 15–20):

```
const TOOL_LABELS: Record<string, string> = {
  chatgpt: "ChatGPT",
  copilot: "Copilot",
  claude: "Claude",
  other: "Other",
};
```

`client/pages/SubmitDeclaration.tsx` (lines 14–19):

```
const TOOL_LABELS: Record<string, string> = {
  chatgpt: "ChatGPT",
  copilot: "Copilot",
  claude: "Claude",
  other: "Other",
};
```

`client/pages/UsageHistory.tsx` (lines 27–32):

```
const TOOL_LABELS: Record<string, string> = {
  chatgpt: "ChatGPT",
  copilot: "Copilot",
  claude: "Claude",
```

```
  other: "Other",
};
```

Impact: Adding a new AI tool (e.g. "Gemini") requires editing three files. Extracting to a shared module eliminates the risk of inconsistency.

Smell 8: Duplicated UI Pattern — Success/Error Messages

Name: Duplicated Code

Description: The success message (green) and error message (red) display blocks use identical Tailwind class strings and the same `<Show when={...}>` wrapper pattern, copy-pasted across form pages.

Identification method: AI-assisted analysis.

Code snippets:

`client/pages/LogUsage.tsx` (lines 156–167):

```
<Show when={message()}>
  <div class="rounded-md border border-green-200 bg-green-50 px-4 py-3 text-sm
text-green-800 dark:border-green-800 dark:bg-green-950 dark:text-green-200">
    {message()}
  </div>
</Show>

<Show when={error()}>
  <div class="rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm text-
red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200">
    {error()}
  </div>
</Show>
```

`client/pages/SubmitDeclaration.tsx` (lines 143–153):

```
<Show when={message()}>
  <div class="rounded-md border border-green-200 bg-green-50 px-4 py-3 text-sm
text-green-800 dark:border-green-800 dark:bg-green-950 dark:text-green-200">
    {message()}
  </div>
</Show>

<Show when={error()}>
  <div class="rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm text-
red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200">
    {error()}
  </div>
</Show>
```

Impact: A design change to the alert styling (e.g. adding an icon or changing border radius) must be applied in every file. A reusable `<SuccessAlert>` / `<ErrorAlert>` component would centralise the styling.

Smell 9: Duplicated UI Pattern — Loading/Error/Empty State

Name: Duplicated Code

Description: Three data-fetching pages repeat the same trio of conditional rendering blocks for loading, error, and empty states with nearly identical markup.

Identification method: AI-assisted analysis, confirmed by manual comparison.

Code snippets:

`client/pages/Classifications.tsx` (lines 73–91):

```
<Show when={loading()}>
  <p class="text-sm text-muted-foreground">
    Loading classifications...
  </p>
</Show>

<Show when={error()}>
  <div class="rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm text-
red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200">
    {error()}
  </div>
</Show>

<Show when={!loading() && !error() && classifications().length === 0}>
  <p class="text-sm text-muted-foreground">
    No classifications found.
  </p>
</Show>
```

`client/pages/Alerts.tsx` (lines 57–71) — same structure, only the text changes:

```
<Show when={loading()}>
  <p class="text-sm text-muted-foreground">Loading alerts...</p>
</Show>

<Show when={error()}>
  <div class="rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm text-
red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200">
    {error()}
  </div>
</Show>
```

```
<Show when={!loading() && !error() && alerts().length === 0}>
  <p class="text-sm text-muted-foreground">No alerts found.</p>
</Show>
```

Impact: A reusable `<DataState loading={...} error={...} empty={...} emptyMessage="...">` component or custom hook would eliminate this three-block pattern from every list page.

Smell 10: Large Component Mixing Concerns

Name: Large Class / Mixing UI and Business Logic

Description: `client/App.tsx` is a 209-line component that combines five distinct responsibilities: (1) authentication state management and localStorage handling, (2) a raw `fetch` call to `/api/auth/login` that bypasses the `api` module, (3) a login form with its own signals, (4) a navigation bar with role-based items, and (5) a page router via `<Switch>`. The `loginRequest()` function in `App.tsx` duplicates the fetch logic that already exists in the `api` module.

Identification method: Manual code review.

Code snippets:

`client/App.tsx` — raw fetch call (lines 22–35), duplicating logic from `client/lib/api.ts`:

```
async function loginRequest(email: string, password: string) {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!res.ok) {
    const body = await res.json();
    throw new Error(body.message || "Login failed");
  }
  const body = await res.json();
  localStorage.setItem("token", body.token);
  return body.user as User;
}
```

`client/App.tsx` — onMount session restore (lines 56–81):

```
onMount(async () => {
    const token = localStorage.getItem("token");
    if (token) {
      const res = await fetch("/api/auth/me", {
        headers: { Authorization: `Bearer ${token}` },
      });
      if (res.ok) {
        const u = (await res.json()) as User;
```

```
          setUser(u);
          setPage(u.role === "admin" ? "classifications" : "log");
          setLoading(false);
          return;
        }
        localStorage.removeItem("token");
      }
      if (import.meta.env.DEV) {
        try {
          const u = await loginRequest("student@ntnu.no", "password123");
          setUser(u);
        } catch {
          // Seed user doesn't exist yet
        }
      }
      setLoading(false);
    });
```

Impact: Any change to authentication logic (e.g. refresh tokens, session expiry) requires modifying the root component. Extracting auth into a dedicated module or context provider, and the login form into its own component, would improve maintainability and separation of concerns.

Smell 11: Double Fetch on Mount

Name: Speculative Generality / Redundant Code

Description: In `UsageHistory.tsx`, `fetchLogs()` is called twice on initial render — once by `onMount` and once by `createEffect` which tracks the `filter` signal. Since `createEffect` runs immediately when the component mounts (tracking the initial value of `filter()`), the `onMount` call is redundant.

Identification method: AI-assisted analysis.

Code snippet:

`client/pages/UsageHistory.tsx` (lines 54–60):

```
onMount(fetchLogs);

createEffect(() => {
    // Track the filter signal so the effect re-runs when it changes
    filter();
    fetchLogs();
});
```

Impact: Every time the UsageHistory page mounts, two identical API requests are made to `GET /api/logs`. Removing the `onMount(fetchLogs)` line would fix the double-fetch since `createEffect` already handles the initial load.

Smell 12: Duplicated Test Helper — `login()`

Name: Duplicated Code

Description: An identical `login()` helper function is defined in all 5 test files. The function has the same signature, same default parameters, same implementation.

Identification method: AI-assisted analysis.

Code snippets:

`tests/auth.test.ts` (lines 5–13):

```
async function login(email = "student@ntnu.no", password = "password123") {
  const res = await app.request("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  const body = await res.json();
  return body.token as string;
}
```

`tests/logs.test.ts` (lines 7–14):

```
async function login(email = "student@ntnu.no", password = "password123") {
  const res = await app.request("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  const body = await res.json();
  return body.token as string;
}
```

Same function repeated in `tests/logs-read.test.ts` (lines 15–23), `tests/declarations.test.ts` (lines 17–25), and `tests/risk.test.ts` (lines 10–18).

Impact: If the login endpoint changes (e.g. new field, different URL), all 5 files need updating. A shared `tests/helpers.ts` module exporting `login()` and `createLog()` would centralise this.

Smell 13: Inconsistent Test Isolation

Name: Inconsistent Style / Shotgun Surgery

Description: Test data cleanup is inconsistent across test files. `tests/risk.test.ts`, `tests/declarations.test.ts`, and `tests/logs-read.test.ts` use `beforeAll` to delete data before

running, but `tests/logs.test.ts` and `tests/auth.test.ts` do not. This means test results can depend on execution order when tests run against the same database.

Identification method: Manual code review.

Code snippets:

`tests/declarations.test.ts` (lines 7–12) — has cleanup:

```
beforeAll(async () => {
  await db.delete(alerts);
  await db.delete(classifications);
  await db.delete(declarations);
  await db.delete(logs);
});
```

`tests/logs.test.ts` — no `beforeAll`, no cleanup. The file starts directly with `describe("POST /api/logs", () => {`.

`tests/auth.test.ts` — no `beforeAll`, no cleanup. Relies on seeded data existing but does not reset state.

Impact: If `tests/logs.test.ts` runs after `tests/risk.test.ts`, leftover data from risk tests could affect log test assertions that check counts. A consistent cleanup strategy (either per-file `beforeAll` or a global test setup) would prevent flaky tests.

Smell 14: Inline Type Definitions

Name: Primitive Obsession / Duplicated Code

Description: Response types are defined inline in multiple places instead of being imported from a shared location. The project generates `client/lib/api-types.d.ts` from the OpenAPI spec, but neither the frontend pages nor the test files use these generated types. Instead, each file defines its own local type.

Identification method: AI-assisted analysis.

Code snippets:

`tests/risk.test.ts` (lines 6–7):

```
type Classification = { id: number; studentId: number; assignmentId: number;
riskLevel: string; undeclaredTools: string[] | null; declaredNotLogged: string[] |
null; createdAt: string };
type Alert = { id: number; classificationId: number; studentId: number;
assignmentId: number; riskLevel: string; createdAt: string };
```

`tests/logs.test.ts` (line 4):

```
type LogEntry = { id: number; tool: string; taskTypes: string[]; assignmentId:
number; createdAt: string };
```

`client/pages/Classifications.tsx` (lines 12–20):

```
type Classification = {
  id: number;
  studentId: number;
  assignmentId: number;
  riskLevel: string;
  undeclaredTools: string[] | null;
  declaredNotLogged: string[] | null;
  createdAt: string;
};
```

`client/lib/api.ts` (lines 23–25) — long inline type annotation:

```
createLog(data: { tool: string; taskTypes: string[]; assignmentId: number }) {
    return apiFetch<{ id: number; tool: string; taskTypes: string[]; assignmentId:
number; createdAt: string }>("/api/logs", {
```

Impact: If a response shape changes (e.g. adding a field), every inline definition must be updated manually. Importing types from the generated `api-types.d.ts` or a shared `types.ts` would provide a single source of truth.

Summary

Coverage

The review covered all source files in the project:

Layer	Files reviewed	Smells found
Backend routes	5 files (`index.ts`, `auth.ts`, `logs.ts`, `declarations.ts`, `classifications.ts`, `alerts.ts`)	5
Backend lib/db	3 files (`auth.ts`, `schema.ts`, `client.ts`, `seed.ts`)	(included in backend smells)
Frontend pages	5 files (`App.tsx`, `LogUsage.tsx`, `UsageHistory.tsx`, `SubmitDeclaration.tsx`, `Classifications.tsx`, `Alerts.tsx`)	6
Frontend lib	2 files (`api.ts`, `utils.ts`)	(included in frontend smells)
Tests	6 files	3

Layer	Files reviewed	Smells found
Total	**21 files**	**14 code smells**

Key Findings

The most prevalent issue is **code duplication** (smells 1, 6, 7, 8, 9, 12, 14), accounting for half of all findings. This is characteristic of AI-generated code, which tends to produce self-contained files by repeating logic rather than creating shared abstractions.

The most impactful issue is **missing role-based authorization** (smell 4), which is a security defect allowing students to access admin-only data.

The most architecturally significant issue is the **god function** (smell 2), where the risk classification engine combines four requirements into one function with no separation between data access and business logic.